

Document 1:

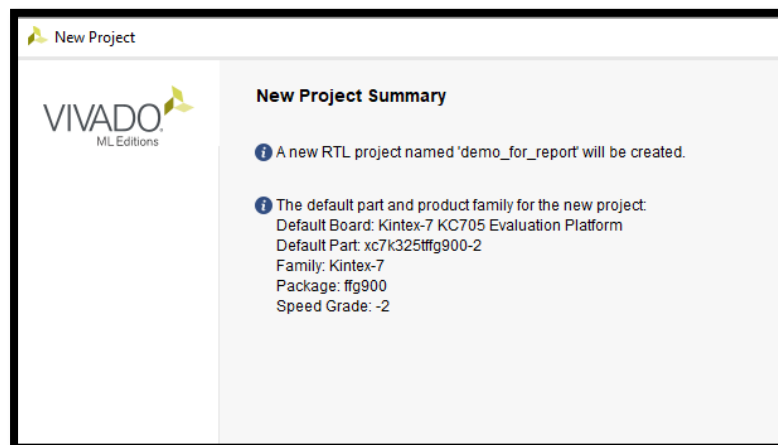
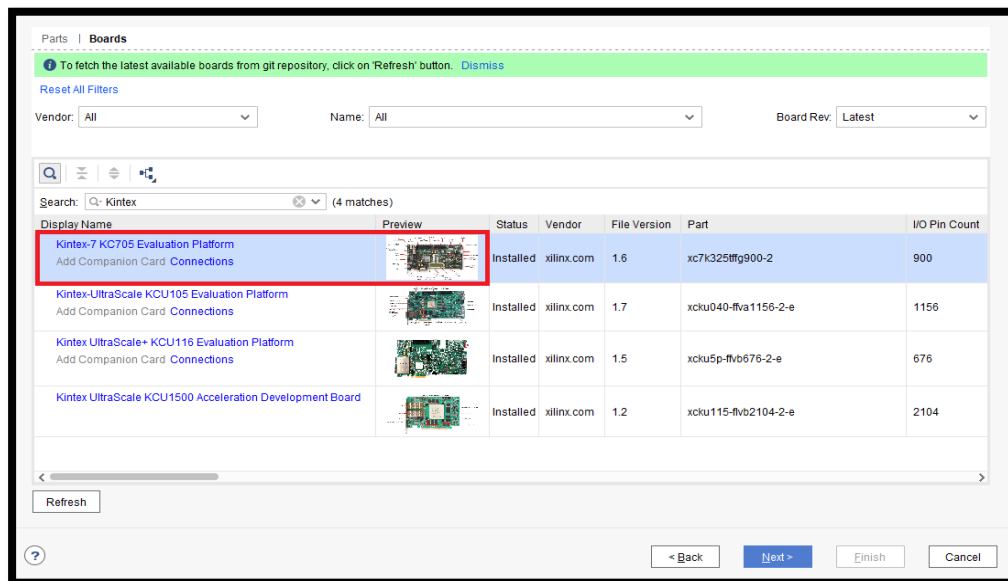
Tool Flow & File Generation

KAPISH GUPTA

Vivado 2022.2 Tool Flow

Step 1: Select your board

For MicroBlaze



For Zynq

New Project

Default Part

Choose a default Xilinx part or board for your project.

Parts | Boards

To fetch the latest available boards from git repository, click on 'Refresh' button. [Dismiss](#)

[Reset All Filters](#)

Vendor:

All

 Name:

All

 Board Rev:

Latest

Search:

Q: zynq

 (10 matches)

Display Name	Preview	Status	Vendor	File Version	Part
Zynq 7000 ZC702 Evaluation Board Add Companion Card Connections		Installed	xilinx.com	1.4	xc7z020clg484-1
Zynq 7000 ZC706 Evaluation Board Add Companion Card Connections		Installed	xilinx.com	1.4	xc7z045ffg900-2
Zynq UltraScale+ ZCU102 Evaluation Board Add Companion Card Connections		Installed	xilinx.com	3.4	xczu9eg-ffvb1156-2-e

Refresh

?

< Back

Next >

Finish

Cancel

New Project

VIVADO_{ML Editions}

New Project Summary

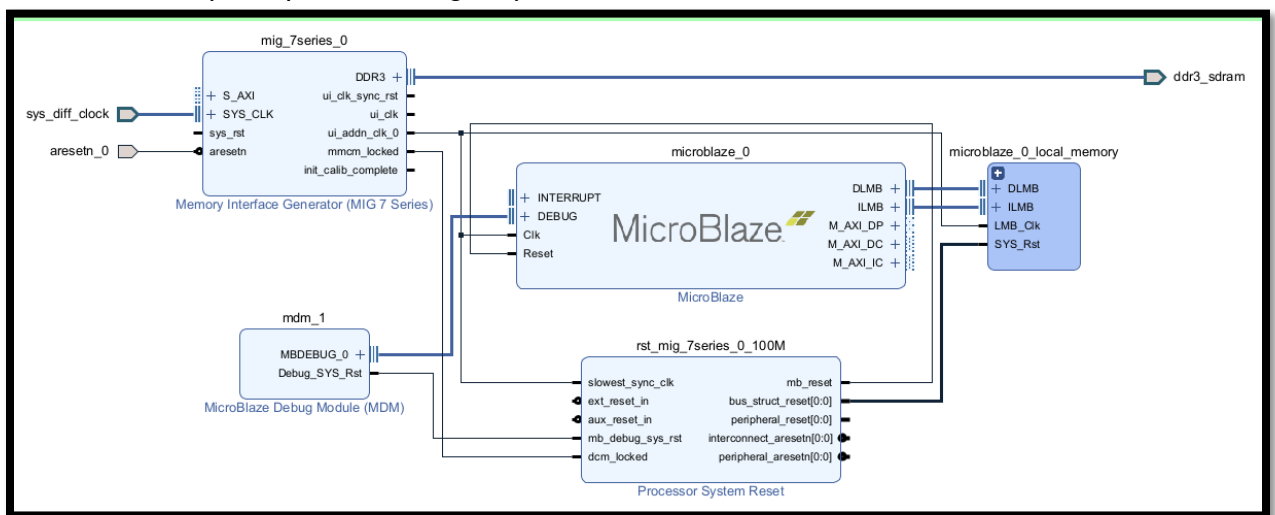
A new RTL project named 'project_2' will be created.

The default part and product family for the new project:
Default Board: Zynq 7000 ZC706 Evaluation Board
Default Part: xc7z045ffg900-2
Family: Zynq-7000
Package: ffg900
Speed Grade: -2

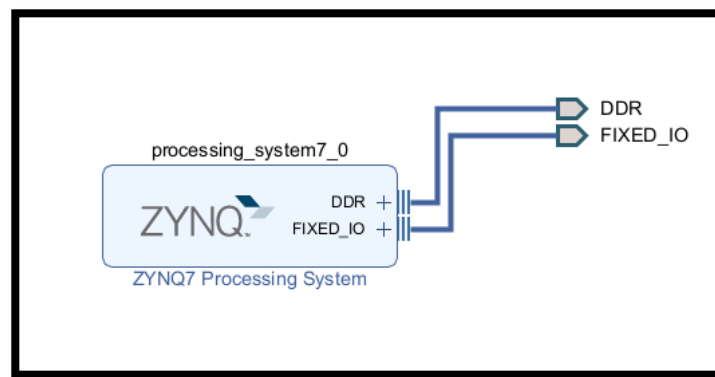
Step 2: Create Block Design

- Required Files: Xilinx IP Catalog definitions.
- Generated File: .bd (Block Design File).
- Function: Acts as the master visual schematic. It connects the physical hardware IP blocks (MicroBlaze processor, Zynq PS, AXI DMA, Xilinx FFT, AXI Timer etc.)

For MicroBlaze (example block diagram)

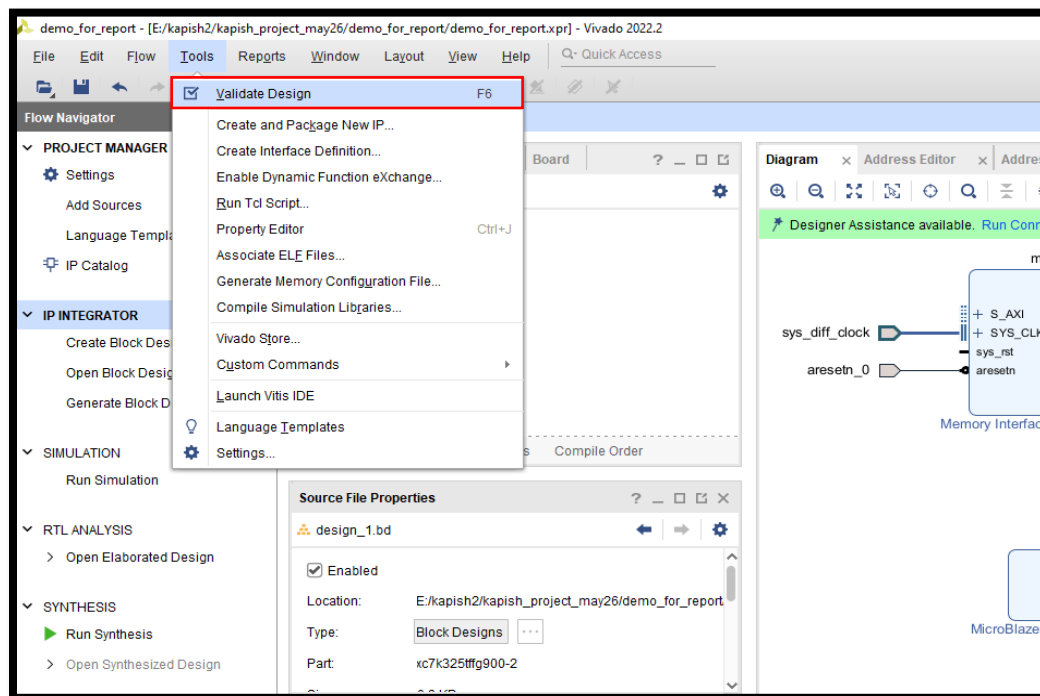


For Zynq(example block diagram)



Step 4: Validate the block diagram

- Function: Vivado runs an automated Design Rule Check (DRC) on the visual schematic. It checks that all connected AXI bus interfaces have matching clock frequencies, that data widths align, and that reset polarities (Active-Low vs. Active-High) are correctly wired.
- Importance: This catches critical architectural errors instantly before spending time in the compilation phases. [Insert Screenshot: The "Validation Successful" pop-up box]

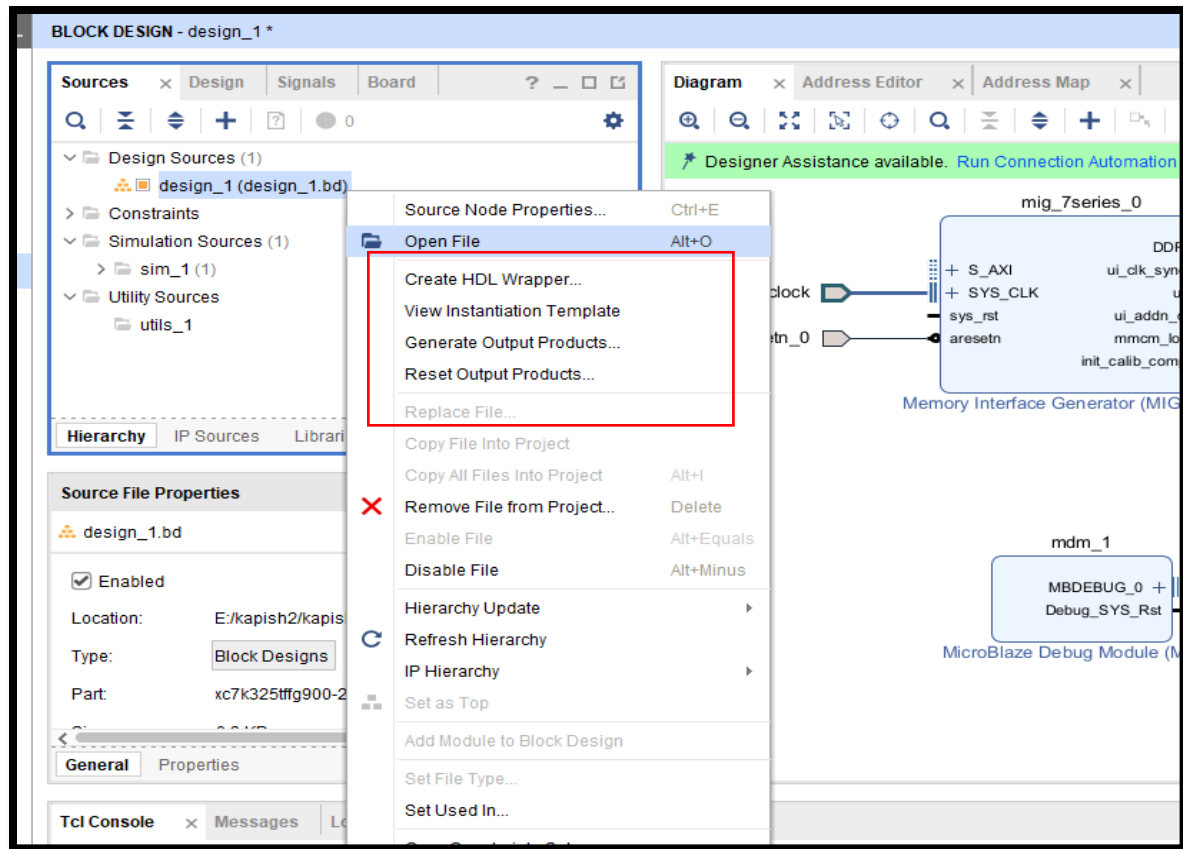


Step 4: Generate Output Products

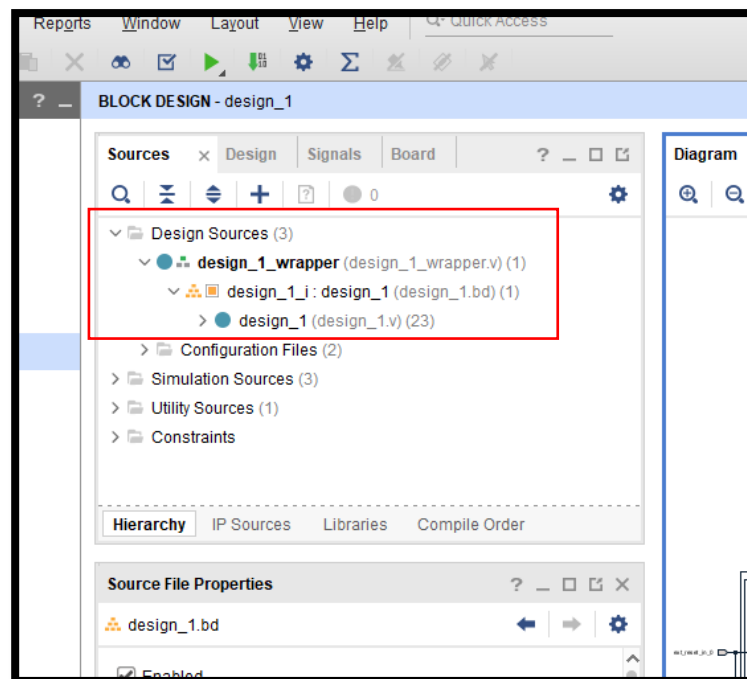
- Required File: **.bd** (Block Design File).
- Generated Files: **.v** / **.vhd** (RTL Source Files) and **.xci** (IP Core Configuration Files).
- Function: Translates the visual schematic into synthesizable Verilog or VHDL code for each individual IP block.

Step 3: Create HDL Wrapper

- Required Files: **.bd** file and generated RTL sources.
- Generated File: **_wrapper.v** or **_wrapper.vhd** (Top-Level HDL File).
- Function: Wraps the entire block design inside a single text-based code module. It formally defines the external I/O pins (e.g., DDR RAM, UART) so they can be mapped to the physical pins on the FPGA board.



After **Generating output products** and **creating the HDL wrapper**, we get,



Step 4: Add constraints

- This requires .xdc file(Xilinx design constraint)
- Function: This file maps your external ports to physical board pins. This file contains the information about the Pin mapping, Clock frequency, IO pins/ IO standards etc.

Step 6: Run Synthesis

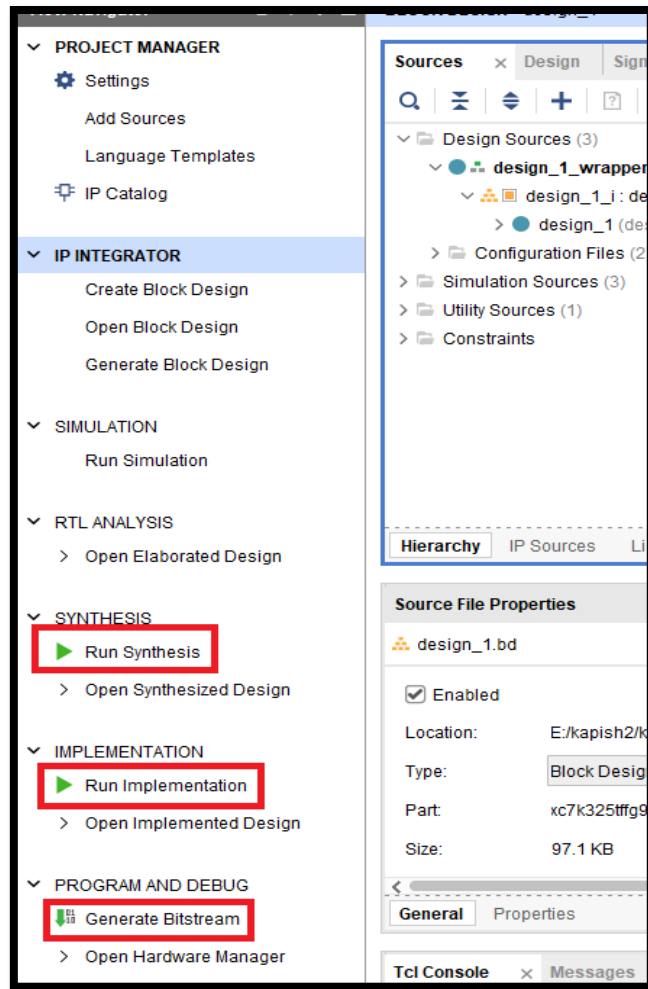
Synthesising a design converts RTL logic to Netlist.

Step 3: Run Implementation

Implementation maps logic gates to specific physical location on the board. It performs Placement and Routing.

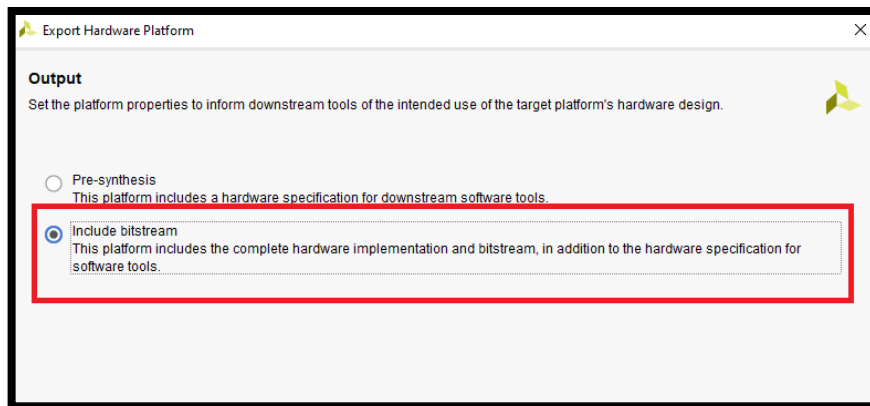
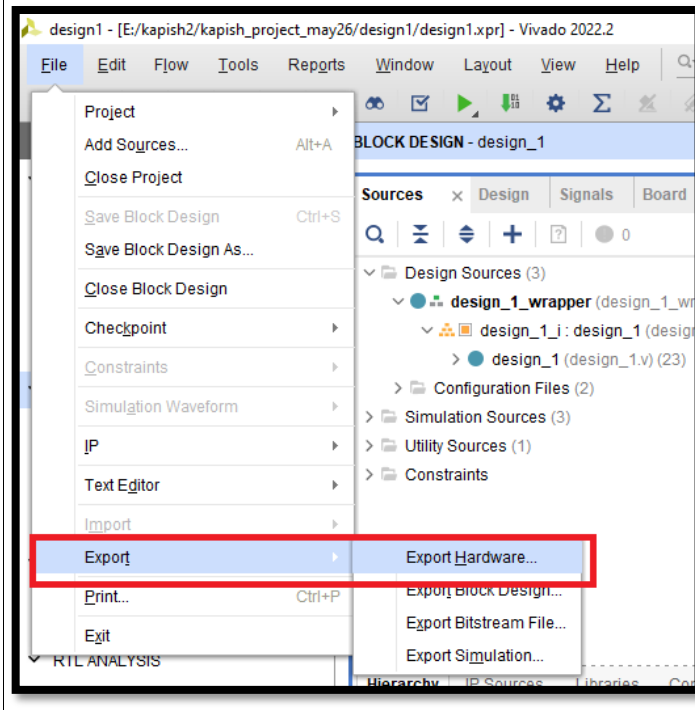
Step 3: Generate Bitstream

- Required Files: _wrapper.v and .xdc (Physical Constraints File, if external pins are used).
- Generated File: .bit (Bitstream File).
- Function: writes the final binary configuration file.



Step 3: Export Hardware

- Required Files: .bit (hardware configuration file) and .mmi(memory map info)
- Generated File: .xsa (Xilinx Shell Archive).
- Function: Packages the hardware bitstream and the system memory map (defining the clock frequency ,AXI addresses of the DMA/FFT/ Timer/UART and other peripherals) into a single archive file.
- This exported file will be imported by Vitis Tool.

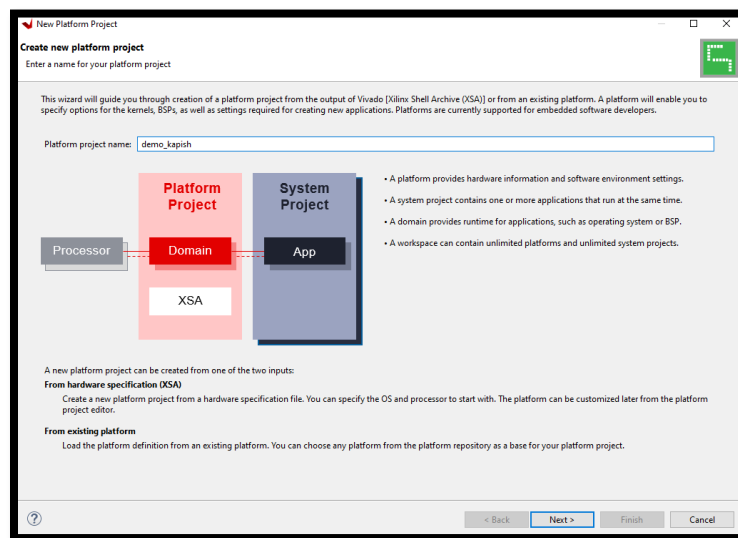


Make sure to include the Bitstream we generated.

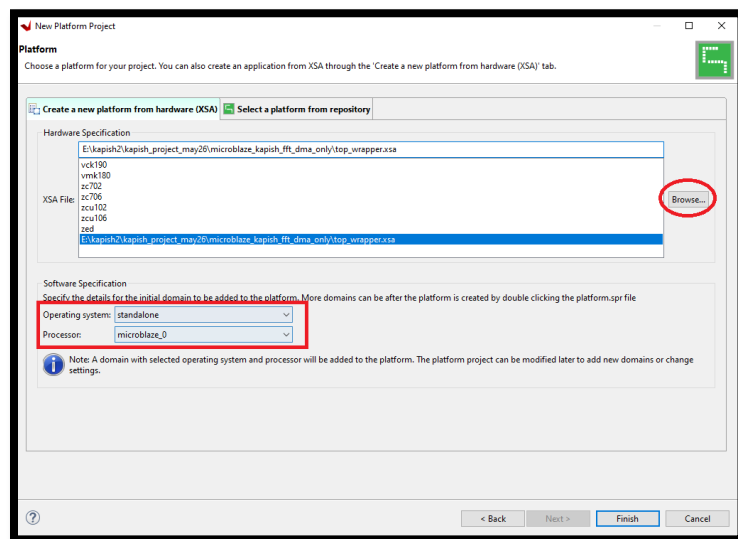
Xilinx Vitis 2022.2 Tool Flow

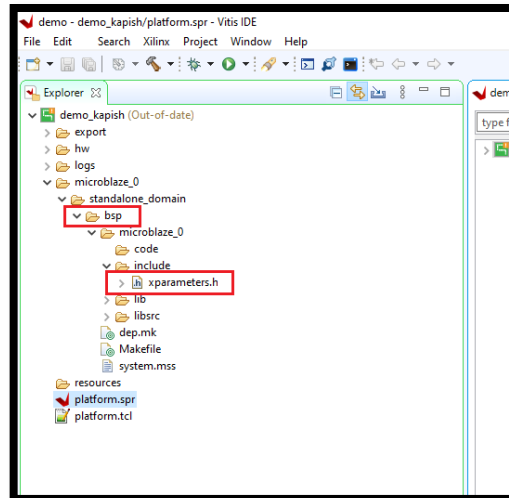
Step 1: Platform Project Creation

- Required File: .xsa (Imported from Vivado).
- Generated Files: Board Support Package (BSP), including xparameters.h and low-level hardware drivers.
- Function: Ingests the hardware architecture and automatically generates the C-code drivers. This creates a software map so the ARM processor knows exactly which memory addresses correspond to the newly created custom hardware.



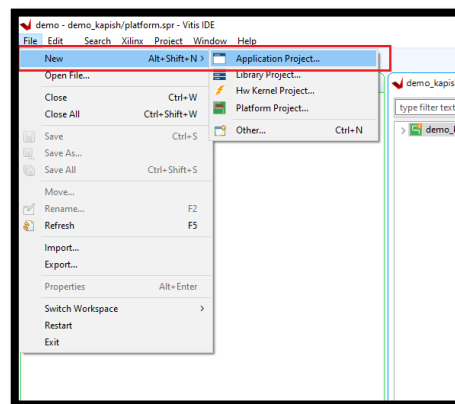
Browse your xsa file.





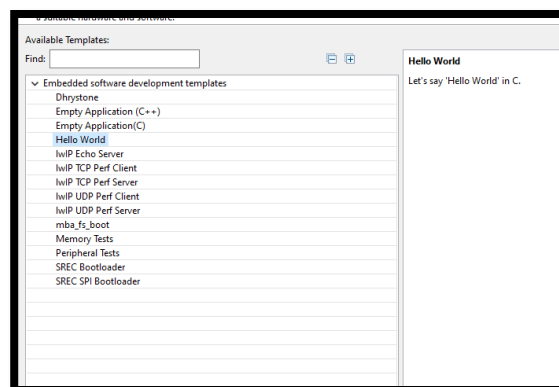
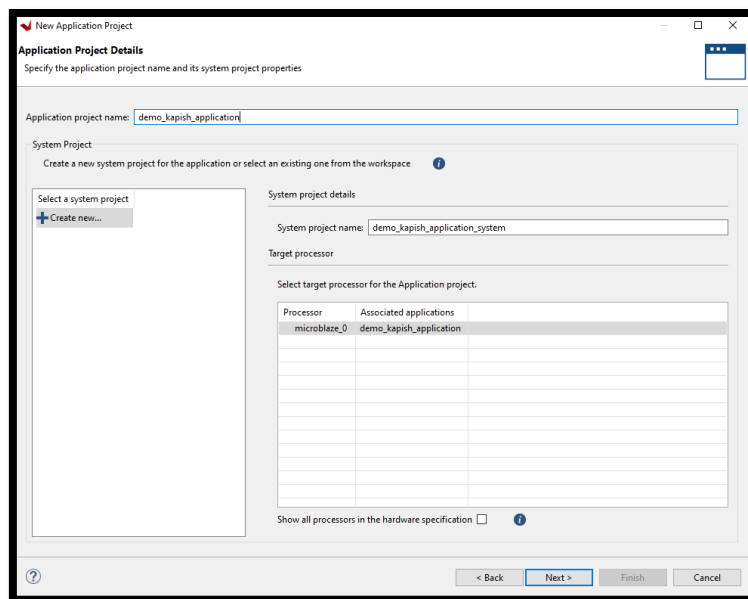
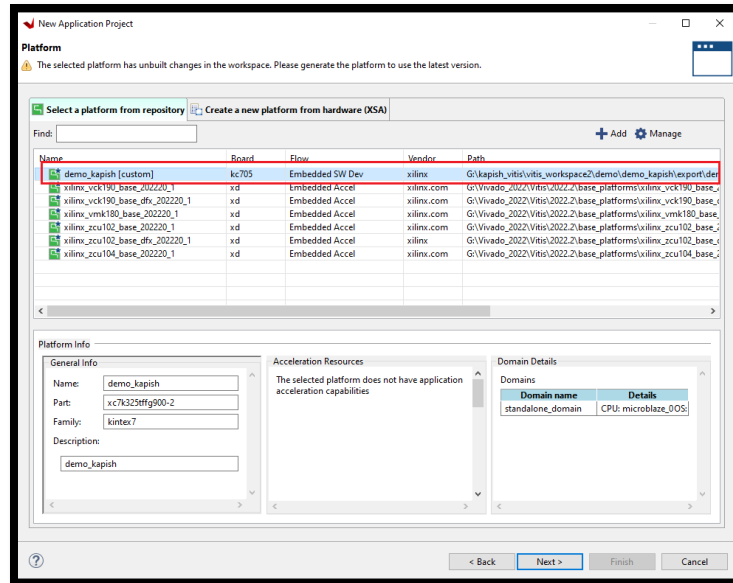
Step 2: Application Project Creation

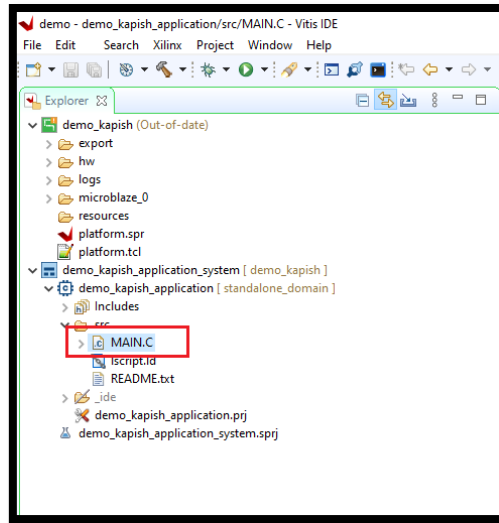
- Required File: Platform Project (BSP).
- Generated Files: .c and .h (Main source files, e.g., helloworld.c).
- Function: Provides the workspace to write the custom bare-metal C code. This is where the DMA control sequences, cache coherency commands, and mathematical logic are defined.



Select your Platform project to make a new application project.

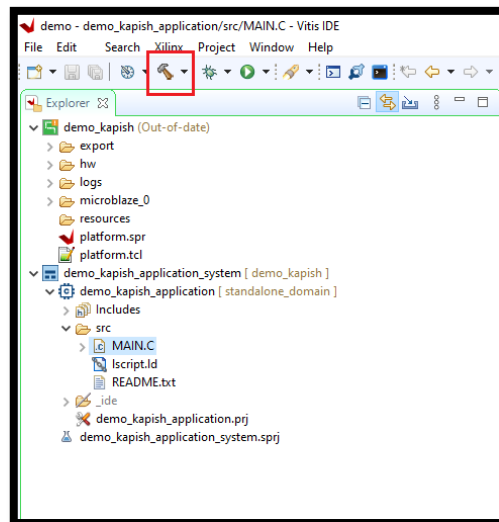
We can make as many application project on single platform project.





Step 3: Application Build

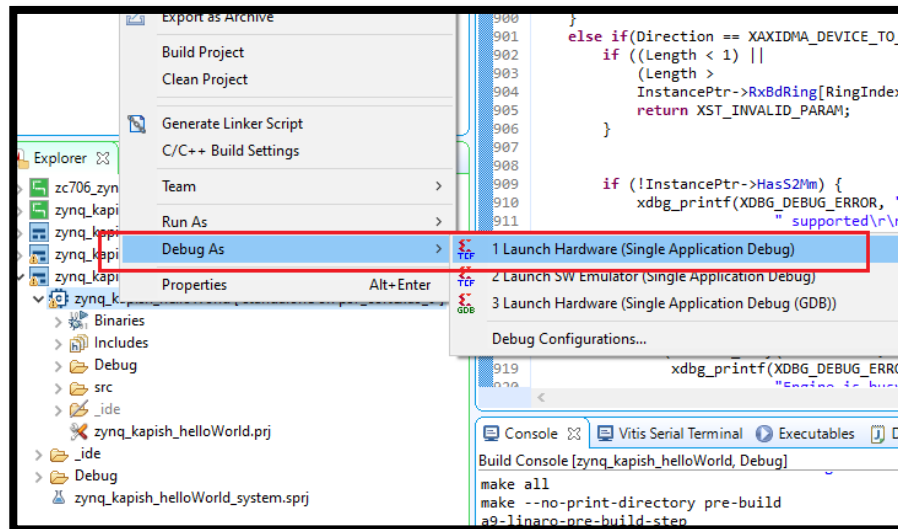
- Required Files: .c source code and BSP libraries.
- Generated File: .elf (Executable and Linkable Format).
- Function: Compiles the C/C++ code and links it with the hardware drivers to create a binary executable file.



Step 4: Launch on Hardware (Deployment)

- Required Files: .bit (Hardware file) and .elf (Software file).
- Generated Files: None (Executes directly on hardware).

- Function: Performs a two-step deployment over the JTAG/USB cable: it flashes the .bit file into the Programmable Logic (PL) to build the physical circuits, then loads the .elf file into the Processing System (PS) memory and commands the ARM processor to begin execution.



Our FPGA is getting programmed.

